研究报告 · 预印本 · 未经同行评议

需求文本如何影响多 Agent 规划的返工

一个可机械校验的编排系统，与 80 次规划运行的对照实验

changkong34@qq.com

2026 年 11 月 6 日 • 数据与代码见附录 B

摘要。多 Agent 编排的收益常被当作前提，但"什么样的任务描述会让编排变坏"很少被测量。 本文搭建了一个可机械校验的编排系统：任务契约把拆解结果写成可检查的字段（写入集、读取集、依赖、 可判定的验收判据），23 条规则在提交前拒绝不合格的计划，执行在独立 worktree 中进行并由独立验收器判定。

在此基础上做了四批共 80 次规划运行，操纵需求文本的详细程度与可锚定性。 主要发现：(1) 需求的详细程度与返工次数呈 U 形，不是单调下降；(2) 80 次运行中 55 次拒稿，错误码 100% 是"判据不可判定"（E014），没有出现第二种失败机制；(3) 用配对实验证伪了"文本长度"和"指称风格"两个解释；(4) 锚点数量存在最优点：0 个显著最差（p = 0.031），1 个最好，2 个反而略差。

结论：需求里给一个可锚定的东西就够了。不给必然返工，给两个略有代价，写得更长不会更好。 该结论在模型版本升级后方向仍成立，但绝对水平会漂移。

1 问题

把一件工程任务拆给多个 Agent 并行做，通常的论证是"节省时间"。但工程实践里更常见的失败不是慢，而是做错了却看起来做完了：拆解时两个任务改同一个文件，合并时冲突；一个任务读了另一个还没写完的代码， 基于旧版本做了决定；验收标准写成"体验良好"，执行者无法判断自己是否完成。

这些失败有一个共同特征：它们在代码里，而不在模型里。可以用契约约束、用规则拦截、 用隔离执行避免。这提示一个更窄但更实的问题——

本文的问题

当编排系统已经能把"拆错了"变成"机器能查出来的错"，那么需求文本的写法还会影响什么？ 影响多大？通过什么机制？

之所以值得问：如果写法的影响可以被机械规则吸收，那就不需要研究；如果影响仍然显著， 那么"把需求写清楚"就成了使用这类系统的隐性成本，而这个成本目前没有被计入任何效率讨论。

2 系统

系统不追求吞吐，目标是把"计划的正确性"变成可执行断言。分三层，每层只做一件事。

2.1 契约与校验

拆解结果不是自然语言计划，而是一组任务，每个任务有五个字段：`goal`（做什么）、`writes`（允许写哪些文件）、`reads`（会读哪些）、`deps`（依赖谁）、`acceptance`（验收判据）。

其中 `acceptance` 是全文的核心约束：每条判据必须是可判定的。 判据只有三种合法形式——可以直接运行的命令、命令加期望输出、带锚点的文件断言。 "带锚点"指句子里必须出现可被机器定位的东西：文件名、路径、数字、反引号内的符号。

在 23 条规则里，有四条决定了后续实验的全部结果：

编号	检查	为什么这条重要
`E014`	判据不可判定	判据必须能被机器执行或核对，否则"做完了"没有意义
`E013`	写入集冲突	两个任务写同一文件，合并必冲突
`E019`	读写依赖缺失	B 读 A 要写的文件却不依赖 A，会读到旧值
`E018`	写入集过宽	写 `**`、`.` 等于没写边界

E019 后来被改成自动修复而非报错：因为"B 读了 A 的产物 → B 必须排在 A 后面"是推导， 不是判断，代码能算对的事不该让模型重试。这个改动把一次真实的连续三次失败降到了零。

2.2 执行与验收

每个任务在独立的 git worktree 中执行，一个任务一个分支。执行者退出后先做越界检查： 对比 `git status` 的路径与任务声明的 `writes`，有越界文件直接废掉该次尝试， 不提交、不验收。这一步只看路径不看内容，成本近乎为零，但拦住了最常见的失控形式。

验收由独立的一方完成：只给它判据、diff 和改动后的文件内容，不给执行者的推理过程， 并且要求每条 PASS 必须给出具体证据。看不到推理，是这一层的设计要点——否则验收会退化成"复述执行者的说法"。

3 实验设计

四批实验，都在同一个仓库（一个约 120 行的任务管理应用）上做，只跑规划阶段，不跑执行。 理由：要测的是"需求怎么写 → 计划被拒几次"，执行者不参与这个变量，但会让每次运行从 1 分钟变成 4 分钟。

批次	设计	n	要回答的问题
一	6 任务 × 5 档详细程度	30	写法有没有影响？形状是什么？
二	10 对配对，只改回指方式	20	是不是"长度"或"指代"在起作用？
三	锚点 0 / 1 / 2，模板生成	24	机制是什么？有没有最优值？
四	第一批原文重跑（换版本后）	6	结论依赖模型版本吗？

三批用了完全平衡设计（每个任务都跑过每一档），因此可以把"任务本身的难度"和"写法"分开看。 第二批用配对设计，每对文本前半段逐字相同，只有二次提及不同——一个重复文件名，一个用"这个模块/它"。 第三批用脚本按模板生成，正文与结尾逐字相同，只改开头提到的锚点数量，以保证除自变量外没有差异。

4 结果

4.1 详细程度：U 形，不是单调

档	写法	平均重写	有重写比例	平均耗时
L1	只有目标，不提文件、不给验收	1.00	100%	88 s
L2	提到文件名，仍无验收	0.67	67%	67 s
L3	说清改哪个文件、做什么，不给命令	0.00	0%	44 s
L4	给出验收命令	0.17	17%	42 s
L5	验收命令 + 边界条件 + 禁改范围	0.33	33%	50 s

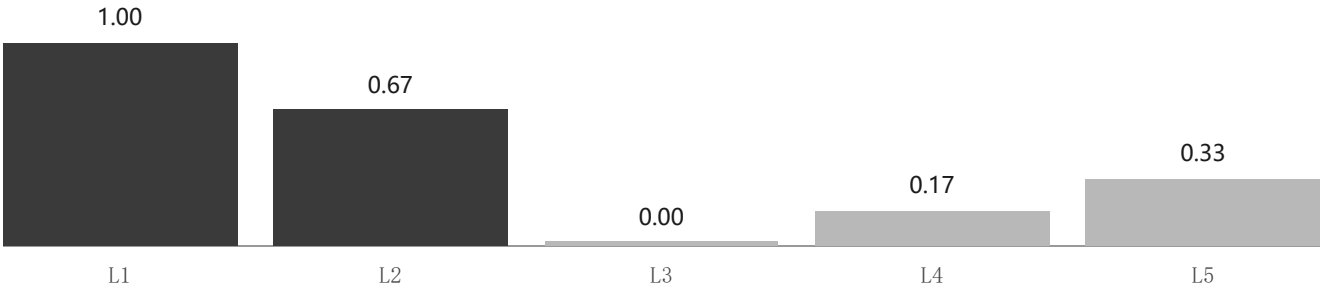


图 1　平均重写次数随详细程度变化。最低点在 L3，L4、L5 回升。

逐任务看也一致：L1 在 6 个任务上是 1.00，L3 在 6 个任务上是 0.00，没有例外。 说明这个形状不是某个任务带出来的。

一个反直觉的点：把验收标准直接写进需求（L4、L5）反而比不写（L3）更差。 后文的机制分析解释了这一点——多写的约束会变成更多条判据，而每条判据都有独立的出错概率。

4.2 代价：含糊让规划成本翻倍

L1 平均 88 秒，L3 平均 44 秒，比值 2.0。多出来的时间全部是重写那一轮。 所以"把需求写清楚"不只是文风问题——在这个系统里它是直接的算力成本。

4.3 配对实验：长度与指代都不是原因

条件	n	平均重写	平均字数
C：重复文件名	10	0.50	105
R：用"这个模块 / 它"回指	10	0.60	88

R 组比 C 组短 16%，但成绩没有系统性差异。

配对差合计 +1，R 更差 3 对、更好 3 对、持平 4 对。这是抛硬币。 关键在 R 组虽然短了 16%，却没有变差——这与第一批"越短越差"的读法冲突，因此排除了"长度"这一解释； 同时指称密度从 0.9 升到 3.4 也没有带来差异，排除了"指代风格"。

4.4 锚点数量：有最优值，且在 1

第三批用模板生成，正文与结尾逐字相同，只改开头提到的锚点数量。

组	锚点数	平均重写	降级次数
A0	0（全文无文件名）	1.62	2 / 8
A1	1（只点名一个文件）	0.62	0 / 8
A2	2（点名两个文件）	1.12	0 / 8

- A0 在 8 个任务里没有一次好于 A1：5 次更差、3 次持平、0 次更好（单尾符号检验 p = 0.031）
- A2 也从未好于 A1：4 次更差、4 次持平（p = 0.062）
- 两次降级全部发生在 A0
- 三档产出的任务数都是 1，因此差异不来自"拆成几个任务"

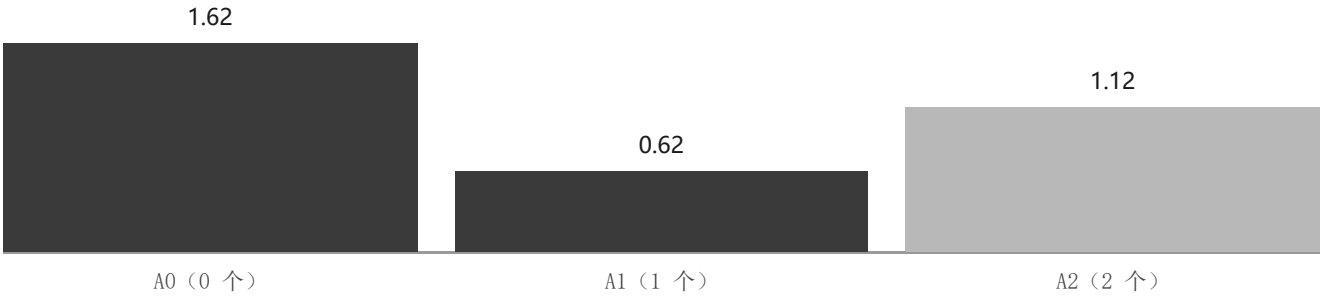


图 2 锚点数量与平均重写次数。最优点在 1。

5 机制

80 次运行，55 次重写，47 次拒稿，错误码 100% 是 E014。没有出现过 E013、E019、E016 中的任何一种。 这个单一性异常干净，也把解释收敛到一点。

> 失败的唯一形式
>
> Planner 把需求翻译成"可验证的判据"这一步会失败。它不是因为不会写代码， 而是因为需求里没有可供锚定的东西时，它只能编出"做好一点""原有行为保留"这类无法核对的句子， 而这些句子会被校验器当场拒绝。

这个机制同时解释了 U 形两侧：

- 左侧上升（L1→L3）：需求里没有锚点，判据只能凭空编。
- 右侧回升（L3→L5）：约束变多，判据条数随之变多。单条判据的出错概率不变， 但"至少错一条"的概率随条数上升。A2 比 A1 差 0.5，也是同一原因——点名两个文件， 就要为两个写入目标各写一套判据。

因此重写风险大致是两个方向的乘积：`f(是否可锚定) × 判据条数`。 两个因子方向相反，所以最优点必然在中间，而不在两端。

6 讨论

6.1 能写什么，不能写什么

可以写	不可以写
含糊需求的返工率显著高于具体需求	具体需求的重写次数是 0
锚点数量存在最优点，1 个优于 0 个（p = 0.031）	A0 的重写次数精确等于 1.62
失败机制单一，全部为判据不可判定	这套规则能消除所有失败

右栏的三条都是把特定版本、特定样本下的数值当作规律。第四批实验正好证明了这一点： 换到新模型版本后，L1 的平均重写 1.00 → 1.00 完全复现，但 L3 从 0.00 变成 0.33。

所以正确的表述是"方向稳、水平漂"：能断言对比关系，不能断言绝对数值。

6.2 定性材料：两种不在规则里的失败

规则之外的失败更值得记。系统稳定运行后仍出现两类，且都来自"契约本身不可执行"，而不是模型能力：

其一，执行者主动停工。某次任务的判据写成"公开函数集合等于修改前的集合"， 而执行者看不到修改前是什么，于是它按提示词里"信息不足就停、不要猜"的约束停下来， 输出"按约束停止"。这不是模型偷懒，是判据引用了一个执行者无法观测的基线。

其二，验收器误判。某次判据是"这段输出受 verbose 条件控制"， 而 diff 里 `if verbose:` 只是三行上下文之一，验收器看不出块结构，判为不通过。 人工复核发现 diff 里确实有该分支——给验收器的信息形态，决定了它能不能判对。

这两类失败都无法用"写集冲突"这类机械规则表达，也不在常见的 Agent 失败分类里。 它们提示：契约的可执行性本身是一个独立的质量维度，与拆解是否正确、代码是否正确都不同。

6.3 一个方法论上的观察

第一批得到的"指称密度与返工正相关（r = +0.44）"看起来是个好结果，但它经不起配对实验。 在那个语料里，短文本必然同时缺少指称词、缺少锚点、缺少验收动词——任何与长度共变的指标都会呈现同样趋势。

这不是数据不够，是设计没有分离变量。第二批之所以有价值，恰恰因为它是个空结果： 它用 20 条配对把两个候选解释同时排除，剩下的解释才站得住。

7 局限

1. 每档 n = 6–8，只能做符号检验。结论方向可信，效应量不可信。
2. 单模型、单仓库、单日完成。第三批跨了模型版本升级（`0.166 → 0.167`）， 绝对水平不可与前两批直接比较。
3. 只测了规划阶段。执行成败、合并冲突、返工后的收敛速度都未纳入。
4. 第二、三批的文本由模板生成，可能有模板效应：模型可能因为句式一致而表现更稳， 而不是因为变量本身。
5. 指称密度与锚点数在本设计下无法分离——用指代替换文件名必然同时减少锚点。 这两者只能作为一个整体讨论。

8 下一步

1. 补样本：A0/A1/A2 各扩到 15 条，把 p = 0.062 压到 0.05 以下。约 40 分钟。
2. 推执行阶段：现在测的是"计划会不会被拒"，还没测"计划执行成不成功"。 后者需要执行者参与，每条从 90 秒变成 4 分钟。
3. 把"契约可执行性"做成检查项：§6.2 那两类失败目前没有规则覆盖， 而它们都是可检测的（判据引用不可观测的基线；验收器信息不足）。

附录 A　全部数据

A.1　第一批：详细程度梯度（n = 30）

任务	L1	L2	L3	L4	L5
persist	1	1	0	1	0
search	1	1	0	0	1
stats	1	0	0	0	1
export	1	0	0	0	0
softdel	1	1	0	0	0
priority	1	1	0	0	0

A.2　第三批：锚点梯度（n = 24）

任务	A0（0 个）	A1（1 个）	A2（2 个）
export	1	0	1
paging	3（降级）	0	1
persist	2	1	1
priority	3（降级）	1	2
search	1	1	1
softdel	1	0	0
stats	1	1	2
validate	1	1	1

A.3　第四批：版本漂移校准（n = 6）

任务	L1 旧版本	L1 新版本	L3 旧版本	L3 新版本
persist	1	1	0	0
search	1	1	0	1
stats	1	1	0	0

附录 B　复现

代码 2,674 行，57 个单元测试。四批实验的语料与结果均为 CSV，可直接重跑。

路径	说明
`orch/`	契约校验、规划器、worktree、执行适配、验收器、编排循环（2,674 行）
`experiments/intents.json`	第一批 30 条（6 任务 × 5 档详细程度）
`experiments/intents_ref.json`	第二批 20 条（10 对，回指 vs 重复文件名）
`experiments/intents_anchor.json`	第三批 24 条（锚点 0/1/2，脚本生成）
`experiments/intents_calib.json`	第四批 6 条（第一批原文，用于版本校准）
`experiments/results*.csv`	四批结果：重写次数、尝试次数、错误码、耗时
`experiments/FINDINGS.md`	逐批结论与局限
`scripts/run_experiments.py`	跑一批，逐条写 CSV，中断不丢已跑结果
`scripts/intent_features.py`	语言学特征 × 返工次数
`scripts/sandbox_check.py`	执行环境自检（写 / git / 越界三项）

```
# 跑一批（以第三批为例）
python scripts/run_experiments.py \
  --file experiments/intents_anchor.json \
  --out experiments/runs_anchor \
  --csv experiments/results_anchor.csv

# 出表
python scripts/intent_features.py --runs-dir experiments/runs_anchor

# 单元测试
python -m unittest discover -s tests -t .
```

数据采集于 2026 年 11 月 6 日，单机（Windows 11，Python 3.13），模型为 codex-cli `0.166.0-alpha.6`（第一、二批）与 `0.167.0-alpha.3.2`（第三、四批）。 全部 80 次运行的原始 trace、diff 与验收证据保存在 `experiments/runs*/` 下。